

# Writing: Algorithms and Definitions.

Javier Larrosa

UPC Barcelona Tech

.

- In CS we often write (i.e, communicate) algorithms ...
- ... that we have previously implemented and tested
- Too often authors are too biased by the implementation (and its history)
- Writing the algorithm differs from implementing it
  - Writing is for **communication purposes** (clarity rather than efficiency)
  - Writing is **done a posteriori** (when key ideas are more clear)
  - Writing means **coding your thoughts** (helps you settle down ideas)

# Writing Algorithms: example

## Social Networks

Consider a Social Network (e.g. facebook). We are concerned about its users and who they follow.

## Celebrities

In a social **sub-network** a **celebrity** is a person that everybody knows, but does not know anybody.

## Goal

Find an efficient algorithm to find **celebrities**

# Lets be formal!

## Abstraction of the problem

Think of the network as a **directed graph**  $G = (V, E)$ . A **celebrity** in  $S \subseteq V$  is a vertex  $c \in S$  such that for every  $w \in S$  different from  $c$  we have that  $(w, c) \in E$  and  $(c, w) \notin E$

There are two reasonable data structures for graphs:

- 1 Matrix of booleans
- 2 Adjacency lists

In the following we will assume the Matrix of booleans.

## natural algorithm

The natural algorithm is *quadratic* on the number of people (vertices)

# Quadratic Algorithm

Let  $M[i,j]$  be the sub-graph of interest

**Function** *Celebrity* **is**

```
  for  $i := 1..n$  do  
     $celeb := true; j := 0;$   
    while ( $celeb$  and  $j \leq n$ ) do  
      if  $i \neq j$  and (not  $M[j, i]$  or  $M[i, j]$ ) then  
         $celeb := false$   
      end  
       $j ++;$   
    end  
    if  $celeb$  then return true;  
  end  
  return false;  
end
```

# Efficient Algorithm (ugly)

**Function** *Celebrity* **is**

```
|  $i := 1;$   
| for  $j := i + 1..n$  do  
|   | if  $M[i, j]$  then  $i := j;$   
| end  
| for  $j := 1..n$  s.t.  $j \neq i$  do  
|   | if  $M[i, j]$  or not  $M[j, i]$  then return false;  
| end  
| return true;  
end
```

# Efficient Algorithm (ugly)

## Invariants:

- $i$  may be a celebrity
- $[1..j]$  (except  $i$ ) are not celebrities

**Cost:**  $O(n)$

# Algorithm 2

## Two useful properties

- 1 There is at most one celebrity
- 2 For any pair of vertices  $(u, v)$ , if  $(u, v) \in E$  then  $u$  is not a celebrity, else  $v$  is not a celebrity



## Algorithm 2

**Function** *Celebrity* ( $G = (V, E)$ ) **is**

*Candidates*  $:= V$ ;

**while**  $|Candidates| > 1$  **do**

$(i, j) := Candidates.FetchPair()$ ;

**if**  $(i, j) \in E$  **then** *Candidates.Push*( $j$ );

**else** *Candidates.Push*( $i$ );

**end**

**return** *IsCelebrity*( $G = (V, E), Candidates$ );

**end**

**Function** *IsCelebrity* ( $G = (V, E), v$ ) **is**

**foreach**  $w \in V$  s.t.  $w \neq v$  **do**

**if**  $(w, v) \notin E \vee (v, w) \in E$  **then return** false;

**end**

**return** true;

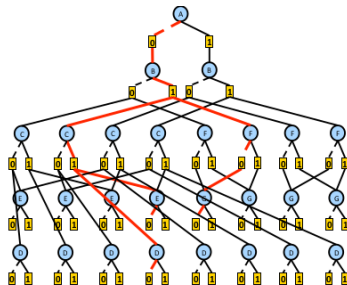
**end**

**Invariants:** *Candidates* set of potential celebrities

# Writing Algorithms: Example

## Search on an AND/OR tree

Find the minimum cost AND/OR solution sub-tree



(b) Example AND/OR search graph. A solution tree is highlighted.

# Example: Complex verbose version (I)

---

**Input:** A graphical model  $\mathcal{M} = (\mathbf{X}, \mathbf{D}, \mathbf{F})$ , pseudo-tree  $\mathcal{T}$ , heuristic function  $h(\cdot)$   
**Output:** Optimal solution to  $\mathcal{M}$

```
1 Create root OR node labeled by  $X_1$  and let the stack of created but not expanded
  nodes  $OPEN \leftarrow \{s\}$ 
2 Initialize  $v(s) \leftarrow \infty$  and the best partial solution tree rooted by  $s$ ,
   $T^*(s) \leftarrow \emptyset, UB \leftarrow \infty$ 
3 while  $OPEN \neq \emptyset$  do
    // Node Expansion
4   Select top node  $n$  in  $OPEN$ 
5   if  $n$  is an OR node labeled  $X_i$  then
6     foreach  $x_i \in \mathbf{D}_i$  do
7       Add AND child  $n'$  labeled  $\langle X_i, x_i \rangle$  to the list of successors of  $n$ 
8       Initialize  $v(n') \leftarrow 0$  and the best partial solution tree rooted by  $n'$ ,
         $T^*(n') \leftarrow \emptyset$ 
9   else if  $n$  is an AND node labeled  $\langle X_i, x_i \rangle$  then
10    foreach OR ancestor  $k$  of  $n$  do
11      Recursively evaluate the cost of the partial solution tree rooted by  $k$ ,
        based on the heuristic function  $h(\cdot)$ , assign its cost to  $f(k)$ 
        // See Algorithm 2
12      if evaluated partial solution is not better than the current upper bound at
         $k$  (e.g.,  $f(k) \geq v(k)$ ) then
13        Prune the subtree below the current tip node  $n$ 
14      else
15        foreach successor  $X_j$  of  $X_i \in \mathcal{T}$  do
16          Add OR child  $n'$  labeled  $X_j$  to the list of successors of  $n$ 
17          Initialize  $v(n') \leftarrow \infty$  and the best partial solution tree rooted by  $n'$ ,
             $T^*(n') \leftarrow \emptyset$ 
```

## Example: Complex verbose version (II)

```
18  Add successors of  $n$  on top of  $OPEN$ 
    // Bound Propagation
19  while list of successors of node  $n$  is not empty do
20      Let  $p$  be the parent of  $n$  if node  $n$  is the root node then
21          return solution:  $v(n), T^*(n)$ 
22      else
23          if  $p$  is an AND node then
24               $v(p) \leftarrow v(p) + v(n), T^*(p) \leftarrow T^*(p) \cup T^*(n)$ 
25          else if  $p$  is an OR node then
26              if the new value of  $v(p)$  is better than the old one (e.g.
27                   $v(p) > c(p, n) + v(n)$ ) then
28                   $v(p) \leftarrow c(p, n) + v(n), T^*(p) \leftarrow T^*(p) \cup \langle X_i, x_i \rangle$ 
29      Remove  $n$  from the list of successors of  $p$ 
      Move one level up:  $n \leftarrow p$ 
```

## Example: Complex verbose version (III)

### Algorithm 2: Recursive Computation of Heuristic Evaluation Function (evalPST)

---

**Input:** Partial solution subtree  $T(n)$  rooted at node  $n$ , heuristic function  $h(\cdot)$

**Output:** Heuristic evaluation function value  $f(T(n))$

```
1 if  $n$  has no successors then
2   if  $n$  is an AND node then
3     return 0
4   else
5     return  $h(n)$ 
6 else
7   if  $n$  is an AND node then
8     Let  $k_1 \dots k_l$  be the OR children of  $n$ 
9     return  $\sum_{i=1}^l \text{evalPST}(T(k_i), h(\cdot))$ 
10  else if  $n$  is an OR node then
11    Let  $k$  be the AND child of  $n$ 
12    return  $c(n, k) + \text{evalPST}(T(k), h(\cdot))$ 
```

---

# Example: Simple version

```
1  Function BBor( $n, ub$ )
2  begin
3      for  $m \in ch(m)$  do
4          if  $c(n, m) + h(m) < ub$  then
5               $ub := c(n, m) + \text{BBand}(m, ub - c(n, m))$ 
6          end
7      end
8      return  $ub$ ;
9  end

10 Function BBand( $n, ub$ )
11 begin
12     if  $ch(n) = \emptyset$  then return 0;
13     foreach  $m \in ch(n)$  do  $q(m) := h(m)$ ;
14     foreach  $m \in ch(n)$  do
15         if  $\sum_{m' \in ch(n)} q(m') \geq ub$  then return  $ub$ ;
16          $q(m) := \text{BBor}(m, ub - \sum_{m' \in ch(n), m' \neq m} q(m'))$ ;
17     end
18     return  $\sum_{m \in ch(n)} q(m)$ ;
19 end
```

# Writing an Algorithms in Documents

- ① Write pseudo-code aiming at communication
  - Avoid being biased by implementation
  - Rethink notation
  - Avoid complex algorithmic structures (e.g. at most two imbricated loops, 10-20 lines per function,...)
  - Use modularity (e.g. functions) to capture important concepts and do not write their code if it is not that relevant.
- ② If the algorithm is important and sufficiently complex, describe it in words in the text (giving meaning to what is going on) showing the connection between the two (see example in next two slides)

---

**ALGORITHM 1:** BT algorithm

---

```
1 BT ALGORITHM( $G, \text{cliqueCost}, \text{mergeCost}$ )
2  $\Pi \leftarrow \text{EnumeratePMCs}(G)$  [7, 17];
3  $T \leftarrow \{\}$ ;
4 foreach  $\Omega \in \Pi$  do
5   foreach  $D \in C(G \setminus \Omega)$  do
6      $S \leftarrow N(D)$ ;
7      $C \leftarrow$  The component of  $G \setminus S$  such that  $\Omega \subset S \cup C$ ;
8      $T \leftarrow T \cup \{(\Omega, S, C)\}$ ;
9   end
10   $T \leftarrow T \cup \{(\Omega, \emptyset, V(G))\}$ ;
11 end
12 sort  $T$  in increasing order of  $|S \cup C|$ ;
13  $dp[(S, C)] \leftarrow \infty$  for all  $(S, C)$ ;
14 foreach  $(\Omega, S, C) \in T$  do
15    $\text{cost} \leftarrow \text{cliqueCost}(\Omega, S)$ ;
16   foreach  $C' \in C(G[C \setminus \Omega])$  do
17      $S' \leftarrow N(C')$ ;
18      $\text{cost} \leftarrow \text{mergeCost}(\text{cost}, dp[(S', C')])$ ;
19   end
20   if  $\text{cost} < dp[(S, C)]$  then
21      $dp[(S, C)] \leftarrow \text{cost}$ ;
22      $\text{optChoice}[(S, C)] \leftarrow \Omega$ ;
```



## 4.2 Detailed Description

Our implementation of the BT algorithm is presented in pseudocode as Algorithm 1. The implementation is mainly based on [7, 17, 23]. As mentioned, the BT algorithm works by decomposing the computation of  $f(G) = f(R(\emptyset, V(G)))$  into the computation of  $f(R(S, C))$  of all blocks of  $G$ . Furthermore, following Corollary 1, the value of  $f(R(S, C))$  is computed as the minimum cost of  $R(S, C)$  with respect to  $\Omega$  over all  $\Omega \in \Pi(G)$  satisfying  $S \subseteq \Omega \subset (S, C)$ .

Algorithm 1 proceeds over triplets of form  $(\Omega, S, C)$ , where  $(S, C)$  is a block and  $\Omega \in \Pi(G)$  satisfies  $S \subseteq \Omega \subset (S, C)$ . The optimal cost of  $R(S, C)$  with respect to  $\Omega$  is computed on Lines 14–18. Whenever this cost is lower than the best known cost for  $R(S, C)$  (Line 20), the value of  $dp[(S, C)]$  is updated (Line 21) and  $\Omega$  is stored in  $optChoice[(S, C)]$  (Line 22). After processing all triplets (Lines 16–22), the value of each  $dp[(S, C)]$  is equal to  $f(R(S, C))$  for all blocks, and  $optChoice[(S, C)]$  contains the PMC  $\Omega \subset S \cup C$  that needs to be completed into a clique when constructing an optimal triangulation of  $R(S, C)$ . Specifically, the value of  $f(G)$  is stored in  $dp[(\emptyset, V(G))]$  (Line 25).

After running Algorithm 1, the optimal triangulation  $H$  can be reconstructed using a breadth-first search like procedure shown in Algorithm 2. Starting from  $B = (\emptyset, V(G))$  (Line 3), the potential maximal clique  $\Omega_B$  corresponding to  $f(R(B))$  is completed into a clique (Line 7), and all blocks  $B_i \in (B : \Omega_B)$  are added to the queue (Lines 8–10). Notice that  $\Omega_B = optChoice[B]$ .

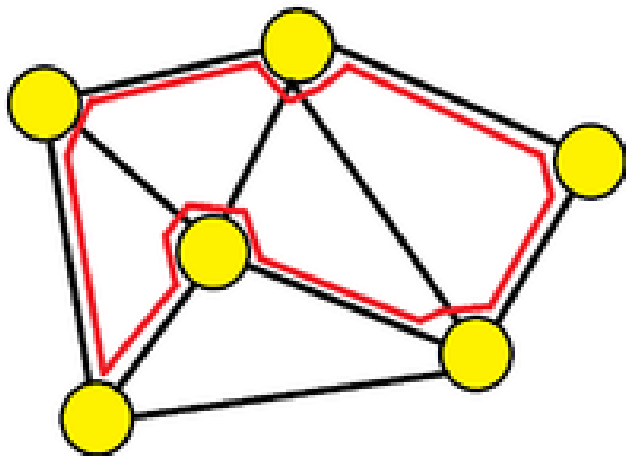
We note that in order to compute the optimal cost of  $R(S, C)$  for a PMC  $\Omega$ , the optimal cost of all blocks in  $(S, C : \Omega)$  needs to be computed. In our implementation of Algorithm 1, all triplets  $(\Omega, S, C)$  are computed before the actual search (Lines 2–10), and then processed in order of increasing sizes of  $S \cup C$  (Line 12). First all potential maximal cliques are enumerated using the procedure from [17] (Line 2) which in turn uses the procedure for enumerating all minimal separators from [7]. Then, for each potential maximal clique, all blocks  $(S, C)$  for which  $S \subseteq \Omega \subset (S, C)$  are initialized (Lines 4–10). The addition of the ‘dummy state’  $(\Omega, \emptyset, V(G))$  on Line 10 is used for retrieving the optimal value  $f(G)$ .

We do not present pseudocode for enumerating potential maximal cliques here, as our implementation is directly based on the pseudocode of [17], using also the optimizations mentioned therein. Let  $G$  be a graph with  $n$  nodes,  $v \in V(G)$  and  $G' = G \setminus \{v\}$ , i.e.,  $G$  with the node  $v$  removed. The enumeration of potential maximal cliques is based on a characterization of  $\Pi(G)$  in terms of  $\Pi(G')$ ,  $\Delta(G')$  and  $\Delta(G)$ . In other words, the set  $\Pi(G) = \Pi(G[V_n])$  is computed by iteratively computing

In Scientific Writing, we often have to write **definitions**

- Definitions can be given in **words** or in **mathematical writing**
- Sometimes we give both
- Sometimes we add an example

# Hamiltonian Graph



# Hamiltonian

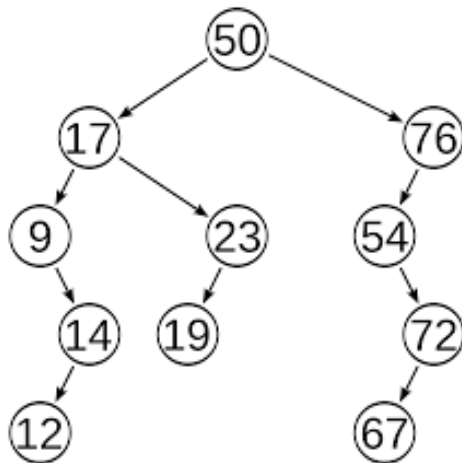
## Hamiltonian

A graph is Hamiltonian if it contains a **loop** such that every vertex in the graph is **visited** exactly once

## Hamiltonian (more formal)

Let  $G = (V, E)$  be an undirected graph with  $V = \{1, 2, \dots, n\}$ . A *path* of length  $k$  is a sequence of vertices  $v_1, v_2, \dots, v_k$  such that  $(v_i, v_{i+1})$  is in  $E$ . A *cycle* is a path that starts and ends at the same vertex (i.e,  $v_1 = v_k$ ). A cycle is *proper* if the only repetition is the starting/ending vertex. We say that  $G$  is *Hamiltonian* if there is a proper cycle of length  $n$

# Binary Search Tree



Assume knowledge on trees (node, left child, right child,...)

## Binary Search Tree

Consider binary trees where each vertex has an associated label. In a *binary search tree* every vertex satisfies that all the labels on its left (respectively, its right) are smaller than (respectively, greater than) or equal to its label.

# Binary Search Tree

## Notation

Consider a binary tree  $T = (V, E)$  with *root*  $r \in V$ . Let  $v \in V$  be an arbitrary vertex. The *sub-tree* on the *left* and *right* of  $v$  are noted  $left(v)$  and  $right(v)$ , respectively. Let's assume that each vertex has associated a *numerical label* noted  $label(v)$ .

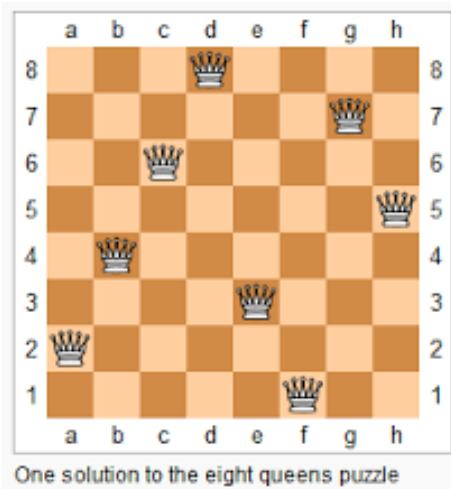
## Binary Search Tree (more formally)

We say that  $T$  is a *binary search tree* if for every vertex  $v$  we have that:

- every label in  $left(v)$  is smaller than or equal to  $label(v)$
- every label in  $right(v)$  is larger than or equal to  $label(v)$

Why did I give a name to the root? :-)

# n-queens





## $n$ -queens

The  $n$ -queens problem consists on placing  $n$  **queens** in an  $n \times n$  chess board in such a way that no pair of queens **attack** each other. Recall that, according to chess rules, queens can **move** any number of positions along rows, columns and diagonals

## $n$ -queens (more formally)

Let  $(x_1, x_2, \dots, x_n)$  be a set of variables taking values in the range  $1..n$ . The  $n$ -queens problem consists on assigning one value to each variable in such a way that for all  $i, j = 1..n$ , with  $i \neq j$ :

- $x_i \neq x_j$
- $|x_i - x_j| \neq |i - j|$

## *n*-queens (balanced)

The *n*-queens problem consists on placing *n* **queens** in an  $n \times n$  chess board in such a way that no pair of queens **attack** each other. We represent that with a set of variables  $(x_1, x_2, \dots, x_n)$  taking values in the range  $1..n$ . Variable  $x_i$  taking value  $j$  represents a queen placed on cell  $(i, j)$ . The problem is to assign one value to each variable in such a way that for all  $i, j = 1..n$ , with  $i \neq j$ :

- $x_i \neq x_j$  (not in the same column)
- $|x_i - x_j| \neq |i - j|$  (not in the same diagonal)

# Round Robin Schedule

Microsoft Excel - IanBalancedRoundRobin\_v3.2.xls

File Edit View Insert Format Tools Data Window Help Adobe PDF

9

Generate

☐ Court or Field Format

☒ Home/Away Format ☒ Use Letters for upto 26 teams

Help

|        | round 1 | round 2 | round 3 | round 4 | round 5 | round 6 | round 7 | round 8 | round 9 |
|--------|---------|---------|---------|---------|---------|---------|---------|---------|---------|
| team A |         | C       | E       | G       | I       | B       | D       | F       | H       |
| team B | I       |         | D       | F       | H       | A       | C       | E       | G       |
| team C | H       | A       |         | E       | G       | I       | B       | D       | F       |
| team D | G       | I       | B       |         | F       | H       | A       | C       | E       |
| team E | F       | H       | A       | C       |         | G       | I       | B       | D       |
| team F | E       | G       | I       | B       | D       |         | H       | A       | C       |
| team G | D       | F       | H       | A       | C       | E       |         | I       | B       |
| team H | C       | E       | G       | I       | B       | D       | F       |         | A       |
| team I | B       | D       | F       | H       | A       | C       | E       | G       |         |

# Round Robin Schedule

## Round Robin

Consider a two-teams sport for which a tournament with  $n$  teams has to be scheduled. A round robin schedule is the pairing of the teams during  $n - 1$  weeks in such a way that any pair of teams are paired exactly once. We can assume  $n$  being an even number, since otherwise we can add a dummy team. Playing against the dummy team represents a resting week

# Round Robin Schedule

## Round Robin (formal)

Let  $n$  be an even natural number. A round robin schedule is a matrix  $M_{n \times n-1}$  such that for all  $1 \leq i \leq n, 1 \leq j < n$ :

- $M(i, j) \in [1..n]$  and  $M(i, j) \neq i$
- if  $M(i, j) = k$ , then  $M(k, j) = i$
- $\forall 1 \leq j < j' < n, M(i, j) \neq M(i, j')$

# Round Robin Schedule

## Round Robin (balanced)

Consider a two-teams sport for which a tournament with  $n$  teams has to be scheduled. A round robin schedule is the pairing of the teams during  $n - 1$  weeks. Let  $n$  be an even natural number (if  $n$  is odd, we can add a dummy team and playing against the dummy team means not playing that week). The schedule is represented as a matrix  $M_{n \times n-1}$  with  $M(i, j) \in [1..n]$  denotes who plays against team  $i$  at week  $j$ . Note that for all  $1 \leq i \leq n, 1 \leq j < n$  the following conditions must hold:

- Teams play teams:  $M(i, j) \neq i$
- Teams are paired: if  $M(i, j) = k$ , then  $M(k, j) = i$
- Same two teams only play once:  $\forall 1 \leq j < j' < n, M(i, j) \neq M(i, j')$

# Induced Width

## Induced Width

Let  $G = (V, E)$  be an undirected graph with  $V = \{1, 2, \dots, n\}$ . The **neighbors** of vertex  $v \in V$  is the set of vertices adjacent to  $v$ . Its **higher neighbors** are those whose index is larger than  $v$ . The **width** of  $v$  is the number of higher neighbors that it has. The **width** of  $G$  is the maximum width among all its vertices.

The **induced graph**  $G^* = (V, E^*)$  is obtained from  $G$  as follows: take its vertices in increasing order. When considering vertex  $v$  connect with a new edge any pair of higher neighbors that are not already connected.

The **induced width** of  $G$ , is the width of its induced graph.

Note that the definition of **induced graph** is **procedural** (that is, algorithmic). In general, it is better not to define things by telling how to compute them. But sometimes it is the most clear way

## Width

Let  $G = (V, E)$  be an undirected graph with  $V = \{1, 2, \dots, n\}$ . The **neighbors** of vertex  $v \in V$ , noted  $N(v)$  is the set of vertices adjacent to  $v$ ,

$$N(v) = \{u \in V \mid (v, u) \in G\}$$

The **higher neighbors** of  $v$ , noted  $HN(v)$  are those whose index is larger than  $v$ ,

$$HN(v) = \{u \in V \mid u > v, (v, u) \in G\}$$

The **width** of  $v$ , noted  $w(v)$  is the number of higher neighbors (i.e,  $W(v) = |HN(v)|$ ). The **width** of  $G$  is the maximum width among all its vertices,

$$W(G) = \max_{v \in V} W(v)$$



## Induced Width

The **induced graph**  $G^* = (V, E^*)$  is obtained from  $G$  as follows (see Algorithm 4): take its vertices in increasing order. When considering vertex  $v$  connect with a new edge any pair of higher neighbors that are not already connected.

The **induced width** of  $G$  is the width of its induced graph  $G^*$ .

**Function** *InducedGraph* ( $G$ ) is

```
 $G^* = (V^*, E^*)$  is a graph;  
 $G^* \leftarrow G$ ;  
foreach  $v \in V$  in increasing order  
  do  
    for  $u, w \in HN(v)$  do  
      if  $(u, w) \notin E^*$  then  
         $E^* \leftarrow E^* \cup \{(u, w)\}$ ;  
      end  
    end  
  end  
return  $G^*$ ;
```

**end**

**Algorithm 1:** Computation of the induced graph  $G^*$  of a graph  $G$ .  $HN(v)$  denotes the set of neighbors of  $v$  with index higher than  $v$  in the graph that is being computed  $G^*$

## 2 Formal:

Game is based on a matrix  $M$  that contains number of cells  $C$ . Each cell  $C$  can have two sets,  $CE$  cell empty or  $CM$  cell with mine. Player one sets the matrix that  $CE+CM=M$ . Player two selects the cell  $C$  that transforms to  $CE$  or  $CM$ . If it is  $CM$  then the game is over. If it is  $CE$  game continues, Some  $CE$  have a hint on them that belongs to set 1,2,3. Hint corresponds to number of  $CM$  in 8 fields around. Strategy for the first player can be to

set matrix  $M$  that  $CM > CE$ . Tactics for the second player can be to reveal hints with a higher number from given set.

# MineSweeper: 2nd ex of bad writing

**Minesweeper** is a two players game in which one player has the role of a **mine layer** and the other of a **mine sweeper**. The game board has a  $n \times n$  size, dividing it in  $n^2$  **cells**. The set of cells is  $C$  and a cell is represented by  $c_{ij}$ , where  $i$  is the row and  $j$  the column, it has a state  $StateC_{ij}$  and a content  $ContentC_{ij}$ , where:

- $StateC_{ij}$  = **discovered** or **unknown** or **flagged**
- $ContentC_{ij}$  = **number** or **mine**; number  $\in [0..8]$

The number indicates how many mined cells are around, in case the cell doesn't have a mine:

---

```
number  $\leftarrow$  0;
for ( $x \leftarrow i - 1$ ;  $x \leq i + 1$ ;  $x++$ ;) do
  for ( $y \leftarrow j - 1$ ;  $y \leq j + 1$ ;  $y++$ ;) do
    if  $ContentC_{xy}$  = mined then
      | number ++;
    end
  end
end
```

---

## MineSweeper: 2nd ex

The mine sweeper can only see  $ContentC_{ij}$  if  $StateC_{ij} = discovered$ , when the game starts the mine layer hides an  $x$  number of mines, and sets all the cells to unknown:  $\forall c \in C, State_c = unknown$ . The mine sweeper has to guess where the mines are, and set  $StateC_{ij} = discovered$ . If he reveals one mine **he loses immediately**. Mine swapper can also **flag** as many cells as  $x$ , number of mines, if he thinks there is a mine. **Mine sweeper wins if:**  $\forall c \in C, State_c = discovered$  or flagged.

# MineSweeper: 3rd ex of bad writing

Minesweeper is a game of two players  $(p_1, p_2)$ . Given a matrix of size  $N \times M$  where  $N, M \in \mathbb{Z}^+$  where each of the cells  $c_{i,j}$  can contain:

- Mine  $m$
- Number of mines nearby ( $n$ )  $n \in \mathbb{Z}^+$  where  $n = \sum_{i,j}^{N,M} (Neight(c_{i,j})) \in m$ .  
We define a cell neighbourhood as  $Neight(c_{i,j}) = c_{i-1,j-1}, c_{i-1,j}, c_{i-1,j+1}, c_{i+1,j}, c_{i+1,j+1}, c_{i+1,j-1}, c_{i-1,j+1}, c_{i,j+1} \parallel 0 \leq i, j \leq N, M$ .

When is  $p_1$  turn a player can:

- Reveal a cell  $reveal_{p_1}(c_{i,j})$  where  $0 \leq i, j \leq N, M$ . This makes propagation when  $\sum_{i,j=0}^{N,M} costCell(c_{i,j}) = 0 \implies reveal(c_{i,j})$ .

# MineSweeper: 3rd ex

- Flag a cell  $flag_{p_1}(c_{i,j})$  where  $0 \leq i, j \leq N, M$  and  $flag(c_{i,j}) \neq 1$
- Unflag a cell  $unflag_{p_1}(c_{i,j})$  where  $0 \leq i, j \leq N, M$  and  $flag(c_{i,j}) = 1$

The game ends when  $p_1$ :

- Uses  $reveal_{p_1}(c_{i,j})$  and  $p_2$  detects that  $c_{i,j} \in m$ .
- $flaggedCells = \sum_{i,j=0}^{N,M} c_{i,j} \in m$  and  $p_2$  verifies  $c_{i,j}$  are correct.

Otherwise the game continues.

Note that if  $c_{i,j}$  is surrounded by  $costCell = 1$  then  $c_{i,j} \in m$ . Similarly per bigger costs.